Ruprecht-Karls-Universität Heidelberg
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

Master's Thesis

TITLE

Name:	Niklas Loeser
Matrikelnummer:	3737339
Betreuer:	Prof. Dr. Artur Andrzejak°
Datum der Abgabe	2024-09-30

I hereby certify that I have written the work myself and that I have not used any sources or aids other than those specified and that I have marked what has been taken over from other people's works, either verbatim or in terms of content, as foreign. I also certify that the electronic version of my thesis transmitted completely corresponds in content and wording to the printed version. I agree that this electronic version is being checked for plagiarism at the university using plagiarism software.

Heidelberg, den 2024-06-20

_____

Niklas Loeser

Abstract

- English -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Abstract

- German -

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

Contents

1 Introduction I
1.1 Motivation I
1.2 Goals I
1.3 Contributions I

2 Background & Related Work 3
2.1 Causal Language Models 3
2.2 Text Generation Strategies 5
2.3 Controllable Generation 6
2.4 Torch 9
2.5 Transformers Library 11
2.6 Code Generation 12
2.7 Language Server Protocol 13
2.8 Related Work 21

3 Design 22
3.1 Interrupt-based meta-strategy 22
3.2 Language Server Protocol aided generation 22
3.3 Dataset design 22

4 Implementation 23
4.1 Interrupt-based meta-strategy 23
4.2 Language Server Protocol aided generation 23
4.3 Dataset design 23

5 Results 24
5.1 Collection method 24
5.2 Possible configurations 24
5.3 Results 24

6 Evaluation 25
6.1 Research Questions 25
6.2 Evaluation difficulties 25
6.3 Evaluation 25

7 Conclusion 26
7.1 Recap 26
7.2 Outlook 26

Acronyms 27

Bibliography 28

Listings 29
List of Figures 29
List of Tables 29
List of Code 29
List of Examples 29

Appendix 30

1 Introduction

Code generation describes the task of generating code for a specific programming language. Generally the code could be generated from a description of the desired functionality, or as a continuation of existing code.

An important use of code generation is to provide code completions in an Integrated Development Environment (IDE). There the completed code removes the need to write boilerplate code.

1.1 Motivation

- Problems:
  - Outdated knowledge and retraining not possible
    - Not flexible enough, as cannot find all needed knowledge for all domains
    - Fine-tuning / retraining slow
  - Dynamic techniques can inject up-to-date information, but are:
    - Relevance: Not specific enough, as they cannot find relevant information
      - Model confusion, as too many nonrelevant tokens are injected and context window is limited
    - Speed:
      - Reliance to terminate, as some iterative methods do not converge
      - Total generated tokens
    - Are not localized, as the model does not know **where** the injected knowledge applies to

1.2 Goals

Goal: Technique to provide the model with up-to-date knowledge while answering the following research questions:

(Reliance over Quality, as quality output does not matter if the fcking program does not terminate)

1. RQ1: How to find relevant information needed for the model generation
2. RQ2: How to provide the model with the relevant information to nudge it towards the desired outcome (understeering causes no change, oversteering is too inflexible, we want nudges)
3. RQ3: How to decrease the performance impact upon the default generation and ensure a reliable generation
4. RQ1: How to find up-to-date information on local code and dependencies
5. RQ2: How to determine the relevancy of the found information on local code and dependencies
6. RQ3:

1.3 Contributions

- General interrupt-based meta-strategy
  - Almost no performance penalty
  - Can provide localized information on-the-fly
  - Robust as it:

  - Gently guides the generation of the model
  - Nudges the model by providing injected context
  - Reduces interrupts
- Implementation of interrupt-based meta-strategy on code generation
  - Programming language independence using LSP
- Creation of custom dataset to evaluate functional and **approach** correctness
  - Automated
  - Plotting
  - Stats collection => Excel
  - Includes pinned dependencies and Python version
  - Implements security concept using Docker

2 Background & Related Work

This chapter provides important background information and related work, necessary to understand the main contributions of this thesis. Firstly, Causal Language Modelss (CLMs) are introduced in Section 2.1. Thereafter, Section 2.2 discusses commonly used text generation strategies.

The section Section 2.3 showcases strategies to modify the text generation of a fixed pretrained CLM. Section 2.5 displays important components of the popular Hugging Face libary, which enables CLM workflows using all of the techniques introduced in the sections before.

Thereafter, Section 2.6 highlights important aspects of code generation. Continuing, Section 2.7 introduces the functionality of the Language Server Protocol (LSP). At last, Section 2.8 introduces previous work.

2.1 Causal Language Models

This section will first introduce the concept of a Language Models (LMs), then move from neural language models, to CLMs. Important text generation strategies for CLM are discussed in depth in the next section.

Language Models

Generally, LMs are probabilistic models of natural language[1]. They predict the next upcoming words based on already seen words[1]:

Previous text	*"The quick brown fox jumps over the lazy _"*
Prediction	*"dog"*

Example 1: Language model predicting the last word of the famous English pangram.

Development on LMs has led to powerful models. These now find widespread use in various applications, such as ChatGPT[2].

Neural Language Models

Neural language models are a type of LM. Their name stems from the McCulloch-Pitts neuron, which is a simplified view of a biological neuron as a computational unit[3,1].

In contrast, a modern neural language model is a network of small computing units. Each of the units is a function that takes a sequence of values and returns a singular output value[1]:

$$f_{\text{unit}} : \mathbb{R}^n \to \mathbb{R} \tag{1}$$

Combining many units into a network allows the computation of a complex function f_{network}. The behavior of f_{network} can be changed by modifying the parameters inside of the network. By applying changes according to feedback, complex functions can be learned.

One very popular architecture for neural language models is the Transformers architecture.

As can be seen in Equation 1, the network is unable to process sequences of natural language directly. The text must first be encoded into numbers, the model can work with. After processing the encoded text, the result can be decoded into natural language once again.

Each neural language model encodes natural language into increasing numbers called tokens using a vocabulary V. The vocabulary is usually fixed, but can be changed as will be shown in the thesis. Each token represents a piece of natural language, such as a

part of a word, or even parts of a sentence. The models initially map natural language to tokens for processing. After processing the tokens these are decoded back into natural language.

Vocabulary	*{"The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "_", ".", "<BOS>", "<EOS>"}*
Text	*"The quick brown fox jumps"*
Tokens ids	*{11, 0, 9, 1, 9, 2, 9, 3, 4}*
Tokens	*{"<BOS>", "The", "_", "quick", "_", "brown", "_", "fox", "_", "jumps"}*

Example 2: Natural language encoded into tokens.

Example 2 showcases the tokenization for the start of the famous English pangram. The tokens values correspond to the index in the given vocabulary. The given vocabulary only provides tokens for full words and does not provide tokens for sub-word fragments. Not only does the vocabulary contain tokens for natural language, but also special tokens. These special tokens encode further semantic information for the CLM. Common special tokens are the Begin of Sequence (BOS), End of Sequence (EOS) and Padding (PAD) tokens:

BOS Marks the beginning of a natural text. Used to delimit several unrelated texts.

EOS Marks the end of a natural text. Used in combination with BOS. The text generation is stopped, when the model predicts the EOS token as the next token.

PAD Used to pad sequences from either side to a fixed length. This is necessary for the model to process several sequences of varying lengths at once.

Definition

Causal language models in turn are a type of neural language model. They rely on the assumption that the probability of a word occuring in a document depends only on the words occuring before it.

This property is called autoregressivity and highly favourable towards text generation, as it enables the continuation of any text. Given a text i_0 as input, the causal language model can predict the next word, as all previous words i_0 are known. After predicting the next word, the word is appended to the input to form the new input i_1. This process is repeated until the model predicts the end of the document using the special EOS token after n steps. i_n then forms the output and the whole generated text.

When applied to text completion or code completion, a program need only return the output i_n without the prefix i_0.

For the purposes of this thesis, a CLM can be defined using the following components:

- Raw next token prediction function f_{rntpf}
- Text generation strategy
- Autoregressive property

The raw next token prediction function is a mathematical function that takes a sequence of tokens and returns the prediction for the next tokens:

$$\begin{aligned} f_{\text{rntpf}}(x) &= u \\ f_{\text{rntpf}} &: \mathbb{N}^n \to \mathbb{R}^{|V|} \end{aligned} \tag{2}$$

The output u has the shape of the vocabulary, where each value u_i represents the logistical probability of token i being the next token. These values are called logits.

In an additional step, the logistical probabilities are normalized into values in the range of $(0, 1)$ to form a probability distribution.

The intermediary step of forming logits provides the benefit, that individual logits can be changed individually without disrupting the probability distribution. This property is essential for the main contributions in this thesis.

2.2 Text Generation Strategies

To generate a full sequence of tokens and thus a document, the model must choose the next probable token in the document using a text generation strategy TGS. After choosing the next token $\hat{w}$, where $1 \leq \hat{w} \leq |\text{vocab}|$, it is appended to the old input x to form x':

$$x' = x^\frown\langle\hat{w}\rangle = (x_1, x_2, ..., x_n, \hat{w}) \tag{3}$$

. Depending on the text generation strategy, the models generate different documents of varying quality. Each text generation strategy has different advantages and drawbacks. The next sections will introduce a few core sampling strategies and highlight the main advantages and drawbacks.

Greedy Decoding

Greedy decoding is the simplest of sampling strategies. The strategy always chooses the token with highest probability. This always yields the current best next token, but not necessarily the best token overall.

Its main advantage is the simplicity. The strategy does not require additional state. Only the current probabilities are needed. This also leads to low hardware requirements, compared to other strategies.

As the current best next token is not often the best token overall, the strategy does not produce the highest quality documents.

$$\hat{w} = \text{argmax}_{w \in V} P(w \mid w_{<n}) \tag{4}$$

Beam Search

Greedy decoding suffers from the problem of always choosing the current most probable token, even if it is not the best token overall. Beam search improves upon this by modeling the decoding process as a search tree. Instead of choosing only a single token, the strategy keeps k possible token choices. These k possible token choices are called hypothesis. At each time step the beam search uses the current k hypothesis to independently continue the generation to create $k * V$ possible token combinations. The strategy then keeps the top-k combinations as new hypothesis.

By modeling the decoding process as a search tree, beam search can choose tokens that are better overall. This leads to a higher quality generation.

But as beam search keeps k hypothesis and decodes $k * V$ possible token combinations, the strategy requires more memory and processing power.

Top-k

Top-k is a generalization of greedy decoding and a sampling strategy. Sampling strategies randomly choose the next token from the probability distribution. Top-k chooses the next token from the top k most probable next tokens. Hence, the model chooses a wider variety of tokens, leading to richer outputs.

As top-k is a generalization of greedy decoding, it has the advantage of being more powerful.

The main drawback is the loss of determinism. Lacking determinism may complicate interpretability methods.

Top-p

Top-k always chooses from the top k most probable tokens, even if most tokens are very unlikely and only few of the k tokens are probable. Top-p allows for more dynamic shapes of probability distributions, by only sampling from the top p percent of tokens.

Temperature sampling

Temperature sampling takes a different approach to top-p. It does not adapt to the shape of the probability distribution, but reshapes it. It divides the raw probability scores by a temperature τ:

$$y = \text{softmax}\left(\frac{u}{\tau}\right) \tag{5}$$

The softmax function normalizes the logits to the range $(0, 1)$, where the sum of all values equals 1. When dividing by $\tau < 1$ all values increase in total. After normalization this skews the probability distribution towards the most probable tokens. In contrast, when dividing by $\tau > 1$ the probability distribution is flattened. This leads to a more uniform distribution.

2.3 Controllable Generation

CLMs suffer from many problems, such as outdated knowledge, insufficient knowledge and insufficient capabilities. Most of these problems can be adressed by fine-tuning the model on further data, such as up-to-date information. The problem with fine-tuning is the need for a large amount of up-to-date data, which might not exist yet. In addition. fine-tuning is resource intensive and must be performed continously, to stay up-to-date.

This section will showcase different ways to control the generation of the model, without the need to perform fine-tuning. The text generation is complex and there are many different strategies and components, which may be controlled. Guided generation will focus in strategies to control the prediction of a single next token, while the chapter prompting strategies will focus on prompting techniques.

Guided Generation

The CLM generates a text, by iteratively choosing the next token ids. The decision on which token ids is the next token id, depends on two factors, as discussed in Section 2.1. It depends on the logits produced through the raw next token prediction function f_{rntpf} and a decoding strategy. Guided generation is a technique to modify the logits before the decoding strategy is applied. The logits are modified according to a set of domain specific rules. The rules not only have access to the current logits, but also to the previously generated text. Thes allows for complex rules.

Consider the following example:

A model is tasked with answering a “yes” or “no” question. The model should only respond with either of both words, without discussing any details or making small talk.
The rules for the guided generation are to set the logits of all tokens to $-\infty$, except when the tokens contribute to a “yes” or “no” answer.
Given the prompt “Is the sun shining?” and the previous tokens “Ye”, the guided generation will set all logits to $-\infty$, except for the token “s”. In the next step only the EOS token will be permitted. Thus, the model will respond with “Yes”.

Example 3: Enforcing "yes" or "no" answers

Guided generation has the benefit of allowing complex rules to be applied to the generation process, to allow a very tightly controlled generation.

The downside of guided generation is, that for complex scenarios, the rules may become very complex and hard to define. In these cases prompting strategies might be more appropriate.

[4]

Prompting strategies

Prompting strategies are a set of techniques to control the generation of the model externally. The model itself, or the model runtime itself is not modified. As such this technique is model agnostic. The general goal is to control the output through structured prompts. In addition, the model may be incorporated into external tooling to enable complex workflows.

The simplest form of control through prompting is to add instructions to the prompt. Models will follow the instructions, if the model was trained on instructions in a similar format. So called "instruction fine-tuned models" were fine-tuned especially on instructions, to improve instruction following capabilities. Non instruction fine-tuned models can still follow instructions, but then the instructions must be chosen carefully.

A useful instruction is to define the desired output format. For example, by specifying that the model should output the response in JavaScript Object Notation (JSON) format or Markdown format, the model will use the specified format. Especially formats such as Markdown and JSON are preferred, as they are common formats and thus the model performs especially well in these formats. Specifying the desired format increases the predictability of the response, which allows automated processing of the response. Specifying to respond in an unknown format does not work. Steering the model does not raise the capabilities of the model, but raises the efficiency with which its capabilities are used.

If the model does not have enough knowledge to respond to a prompt, the prompt itself can be structured to supply the needed information. Given the model has the ability to understand the provided information, it can then act on the information. A popular technique is **few-shot prompting**[5].

Few-shot prompting involves providing the model with several examples in the prompt. The goal is that the model understands the provided examples to learn the pattern and apply it to its response. The learning is undertaken "in context" and not persisted outside of the prompt itself[5]. The knowledge provided to the model is the format of the task and matching response.

Prompt:
Q: What is the capital of Germany?
A: Berlin
Q: What is the capital of France?
A: Paris
Q: What is the capital of Italy?

Response:
A: Rome

Example 4: Few-shot prompting

As can be seen in Example 4, the model prefixes the response with A: and does not provide additional information. The output format was not specified explicitly, but matches the format of the examples perfectly.

In contrast to providing knowledge about a desired response format, **context injection** adds important context into the prompt, which the model then uses to respond.

Prompt:
The capital of Germany was changed to Heidelberg.
What is the capital of Germany?

Response:
Heidelberg

Example 5: Context injection

Example 5 demonstrates how adding information controls the response of the model. This technique is very powerful, but comes with two strong restrictions:
1. Finding relevant context.
2. Limited prompt size.

The first restriction is that the injected context must first be found. This is a hard search problem, as the information must be retrieved **before** prompting, before the intent of the prompt is known yet. In Example 5 the task of the CLM is question answering. Even though the general task is known, the software has no way to known the specific domain, such as "country capitals" beforehand.

The second restriction is the limited prompt size. Models are only capable of focusing on a prompt of limited size. Thus, token space is precious and may not be wasted. Providing the model with an up-to-date version of Wikipedia for every prompt oversaturates the prompt size limit instantly. The injected context must be **targeted** and **relevant**. The more **targeted** the context is, the less tokens are used.

As finding relevant context is a hard problem, several steps are needed to search for the context. Searching for relevant context may even involve the output of another CLM. Strategies that involve the model in a multi-step process are called meta-strategies. These strategies will be further discussed in the next section.

Meta-Strategies

Meta-strategies incorporate a model into an algorithm according to a wider strategy. These strategies shift the focus to viewing the CLMs as a building block to be used as needed. Several models with different strenghts could be combined into a meta-strategy.

Generally, the main benefits are ease of use and flexibility. Any model can easily be integrated into the meta-strategy and stay interchangable. By combining several models into a multi-turn process, including external tooling, powerful behavior is possible.

In contrast, common drawbacks of meta-strategies are the added complexity towards the already complex text generation. In addition, calling models several times increases the runtime of the program, if used without care.

Meta-strategies are applied to e.g. remedy the model lack of up-to-date knowledge. A model may not have the necessary knowledge for a task, or the knowledge it has is outdated. In such cases the meta-strategy can supply up-to-date information and focus the model towards the problem.

Chain of Thought

Chain of Thought is one very central meta-strategy, as it significantly improves the reasoning capabilities of a model[6].

The key idea of chain of thought is to apply the divide and conquer strategy to the text generation process. Instead of the model trying to solve a complex reasoning problem in a single step, the meta-strategy divides the problem into several intermediate step. The model is then able to solve the simpler intermediate steps. Each intermediate step is a thought the model contributes to the final solution. The sequence of intermediate steps forms the "chain" of thoughts[6].

Additionally, the model will use thoughts of previous steps to reason about the next step.

Tree of Thought

Tree of thoughts is a generalization of chain of thought. Chain of thought is only capable of performing linear decision-making[6]. Ultimately, performing complex reasoning is a search problem and thus, linear reasoning is insufficient. The tree of thoughts meta-strategy generalizes the thoughts into a tree structure. Each intermediate problem is then a node in this search tree. The root of the tree is the initial problem. The goal of this search problem then is to find an accepted end node - a finishing thought[7].

By structuring the thoughts into a tree, common tree search algorithms can be applied to the problem. Various algorithms such as a breadth-first search or depth-first search can be applied[7].

In general, the tree of thoughts meta-strategy is more powerful than the chain of thoughts. It is a very powerful and flexible strategy, which even allows following thoughts and then backtracking[7].

[7]

2.4 Torch

Torch is a widely used machine learning library in Python. It enables the creation and manipulation of tensors in different precisions. The Transformers provides high level abstractions which base on Pytorch. This section shortly introduces a few core concepts of Pytorch, which are used in the Transformers library. It will focus on version 2.2° of PyTorch.

PyTorch provides operations on tensors. A tensor is a multi-dimensional array of numerical values and the generalization of a matrix[8]. Each tensor typically only contains values of the same type, such as value from $\mathbb{R}$[8].

One of the applications for tensors is to represent multi-relational data[8]. In that case each dimension represents a single relation.

For CLM inference different types of tensors are used. For example, the token sequence during parallel text generation is a two-dimensional tensors of the following relations: (batch, token). The second dimension represents a single token sequence, while the first dimension enables parallel text generation for n token sequences.

Another example is the next token prediction tensor, which is a two-dimensional tensor of the following relations: (batch, vocabulary). The second dimension contains the probability distribution over the whole vocabulary, while the first dimension enables parallel text generation.

The second example only displays the tensor for a single next token prediction. By adding another dimension for the token relation, the whole probability distribution history could be represented.

After providing the motivation for tensors, the following list will introduce core terminology for use with tensors:

Order Number of dimensions of a tensor
Shape Number of elements in each dimension
Slice Two-dimensional subfield of a tensor. Obtained by fixing all but two indices.
Fiber One-dimensional subarray of a tensor. Obtained by fixing all but one index.
Matrix Two-dimensional tensor.
Vector One-dimensional tensor.
Scalar Zero-dimensional tensor.

[8]

Fixing indices on a tensor will return a subfield or subarray, which in itself is a lower-order tensor. By fixing all indices a scalar is obtained. A slice and fiber are special terms for lower-order tensors obtained from fixing indices on a tensor.

As introduced earlies, tensors usually contain values of the same type. Each PyTorch tensor contains values of a single data type, in short: dtype. PyTorch supports many different numerical data types, of which the following are relevant:

Data Type	dtype
32-bit floating point	torch.float32
16-bit floating point 1	torch.float16
16-bit floating point 2	torch.bfloat16
64-bit integer (signed)	torch.int64
32-bit integer (signed)	torch.int32
Boolean	torch.bool

Table 1: Relevant PyTorch data types

The data types can be used by specifying the dtype parameter when creating a tensor. The value is one of the variables listed in the dtype column in Table 1.

The difference between torch.float16 and torch.bfloat16 is the precision of the floating point number. Both formats use 16 bits to encode the floating point number. The difference lies in the number of bits used to encode the mantissa.

The torch.float16 is a half-precision floating point number, as defined in the IEEE 754 standard. It uses one sign bit, five exponent bits and 10 mantissa bits[9].

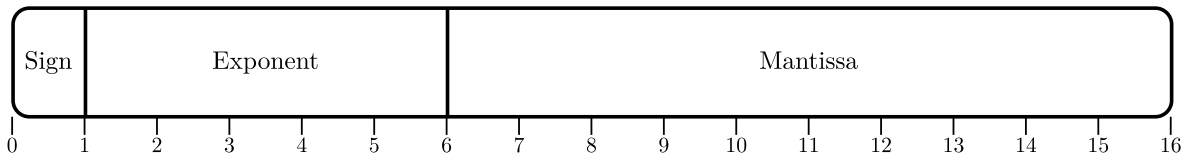


Figure 1: IEEE 754 half-precision floating point number: float16

In contast, the torch.bfloat16 keeps the full range of the full-precision floating point number of 8-bits, but has a reduced precision of 7 mantissa bits[10].

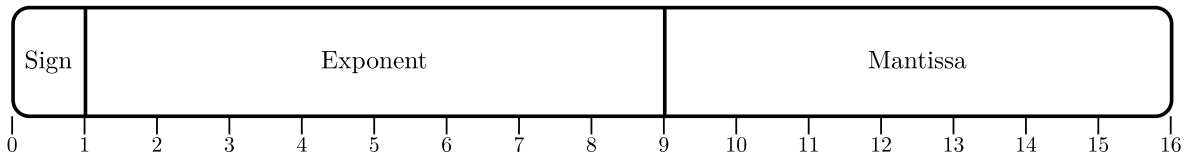


Figure 2: Brain Floating Point Format: bfloat16

The format was introduced by Google Brain and is used in their Tensor Processing Units (TPUs)[10], as the reduced mantissa precision reduces the size needed for physical hardware multipliers[10].

In addition to the precision, another aspect of a PyTorch tensor is the **physical location** it is allocated at, the device. The function torch.device creates a reference to such a physical location. Possible relevant device locations are cpu and cuda, where cpu allocates a tensor on the CPU memory, while cuda allocates the tensor on GPU memory. Already creates tensor can be **moved to a different device** by calling the to method with the new device.

To **create a tensor**, the function torch.tensor is used. It takes data and optionally dtype and the device. The order and shape of the tensor can be accessed through the dim and shape attributes. Indexing the tensor is done through the Python tensor[indice_list] notation, where untouched dimensions are marked through the : symbol, e.g.: tensor[3,:,1]. Iterating the tensor will iterate over the first dimension.

2.5 Transformers Library

Transformers is the name of the popular Python machine learning library developed by HuggingFace. It provides broad support for many different machine learning domains, such as computer vision, audio processing, multimodal models and **natural language processing**. For each domain, the library enables simple training and inference of state-of-the-art models. The thesis heavily relies on functionality provided by this library, as it provides in-depth features to implement the more advanced features. It provides support for all techniques introduced in Section 2.1 and Section 2.2.

As the library receives frequent updates, the thesis focuses on version 4.41.3°. In addition, the library is complex and enables many different use-cases through many abstraction layers. This section will focus on the CLM use-case, omit unneeded arguments and list parameters only available for CLM.

The structure of this section will mirror the structure of previous sections. It will start by introducing CLMs, then continue with tokenizers. Afterwards, the text generation functions are presented.

Using different text generation strategies with the Transformers library will follow. At last, implementing controllable generation will be shown.

Models

In transformers, every type of model architecture is represented by a model class. Each model defines the layers of the model and the operations between layers. Common model functionality is inherited through the base class PreTrainedModel.

Pretrained models provide the following functionality (not exhaustive):

- Loading pre-trained weights
- Resizing the token embeddings

Loading a pre-trained model for inference is done by calling the from_pretrained method on the model class:

```python
from_pretrained(
    pretrained_model_name_or_path,
    torch_dtype: str | torch.dtype = None,
    device_map: str | Dict[str, Union[int, str, torch.device]] = None,
    **kwargs
) -> PreTrainedModel
```

Listing 1: Loading a pre-trained model

Calling Listing 1 returns a model instance with pre-trained weights, loaded onto one or several physical devices. The model is either loaded from a local directory, or from the HuggingFace model hub. Where the model is loaded from depends on the type of the pretrained_model_name_or_path parameter.

The HuggingFace model hub allows for easy access of pre-trained models, given the name of the model and the name of the publisher. The model name has the format publisher_name/model_name. Some[1] of the many available CLM for code generation are:

- codellama/CodeLlama-7b-Instruct-hf
- deepseek-ai/deepseek-coder-6.7b-instruct
- m-a-p/OpenCodeInterpreter-DS-6.7B
- ise-uiuc/Magicoder-S-DS-6.7B

[1]More code generation models can be found in the Bigcode models leaderboard°

In the case that a model name from the HuggingFace hub is provided, the model is downloaded and cached locally. If the model is already present, it is loaded from cache. By default, the model will be loaded using

Config

Models show dynamic behavior, as they can be configured at runtime. This dynamic configuration enables the use of different generation strategies, by setting different model configuration values. Model configurations are classes inheriting from PretrainedConfig. The PretrainedConfig includes configuration values for the model architecture and for the mentioned dynamic behavior.

Tokenizers

Text Generation

Controllable Generation

2.6 Code Generation

Code generation is a task in the general domain of text generation. It refers to the generation of any type of source code. This section will first introduce several important applications for code generation. Then it will list several problems common to specifically code generation. At last general approaches to improve code generation are shown.

Uses

This section will introduce three common use cases, which slightly differ in their requirements. Common requirements are the following:

- The generated code should be syntactically correct.
- The generated code should be semantically correct.
- To increase maintainability, the generated code should follow best practices.
- The developer might be working on a single file or a large project.
- The developer could work on a project of **any** programming language.
- Knowledge about related code:

The CLM performing the task should take the following context into consideration when generating code: At the core, the CLM has the context of the local file that is edited. The next wider context is code in a local repository, when working on a larger project. At last, the model should use knowledge of installed packages.

The first use case is **code completion** for use in IDEs. In this scenario, a developer will have installed tooling to run autocomplete in parallel to his typing. To display the autocompletion before the developer continues writing, the code must be generated in a timely fashion.

The second use case is for generation of **unit tests**. This scenario differs in the size of the generated code. Generated unit tests generally are larger than code generated as inline code completion. Additionally, the generated code

The last use case is for prototyping, in which a single or several files are generated. The difference to other use cases is the increased runtime. Moreover, the code is generated from a task the developer provided.

Problems

Current code generation models and strategies are capable of the following:

- Generating syntactically correct code.
- Generating semantically correct code.
- Generating code for any programming language.

Current techniques still struggle generating code, which is aware of the local code base.

Requirements, which current techniques do not fulfill are topics, such as following best practices and having knowledge about related code.

General approaches

2.7 Language Server Protocol

Many modern editors and IDEs provide features such as:

- Autocompletion
- Signature help
- Go to the definition of a symbol
- Go to the uses of a symbol
- Provide diagnostics data on the current file
- Rename symbols

This list is not exhaustive, and the features between different editors may vary. These features are vital for any programmer, as the effort to create and maintain code is significantly reduced. Navigating large code bases and finding documentation on used code is highly beneficial.

These many features take a significant effort to implement for a **single** programming language and editor. Providing support for several programming languages massively increases the implementation effort. In addition, the created features must be maintained next to the editor itself. As more programming languages are designed and are adopted, the need for tooling for these programming languages arises. Editor maintainers must then integrate these features for the new programming language.

Providing support for an ever increasing amount of programming languages is hard, but supporting these features for an ever increasing amount of programming languages in every editor is impossible. Many diverse editors exist, such as Emacs, VSCode, Neovim (modern fork of Vim), Zed (the successor of Atom), Jetbrains IDEs, Helix and many more.

Thus, a solution is needed to extract thes **common** features from the editors and provide them centrally, such that every editor need only implement an interface to integrate the common features. The Language Server Protocol is an effort to standardize a common feature set for programming language tooling for use in IDEs. The protocol was designed my Microsoft for use in its Visual Studio Code editor, but is now an open standard. This thesis will focus on version 3.17 of the protocol. The protocol includes many different features, of which the following sections introduce a few important. The full specification may be accessed

Design

The protocol is based on a bidirectional client-server architecture. The client is a tool such as the IDE. The server, henceforth called the language server, is programming language specific tooling that provides a subset of the protocol. Both the client and server must not implement the full protocol. The protocol is capability based. Both parties define and communicate their capabilities, then find common capabilities and restrict their use to them. Each client can connect to multiple language servers, and each language server can serve multiple clients. As such a client could use a language server for the Rust programming language while the user is coding on a Rust project. As soon as the user

switches projects to a web project, the editor could then connect to several language servers to provide support for HTML, CSS and JavaScript at once. In that scenario several simultaneous language servers are necessary, as CSS and JavaScript can be embedded into HTML. The communication between the client and server must be bidirectional, as the server must be able to notify the client about events occuring, such as new diagnostic information.

JSON-RPC

The base protocol relies on JavaScript Object Notation-Remote Procedure Call (JSON-RPC). The following section will give a short introduction on the protocol. The content part is specified using JSON-RPC. JSON-RPC is a light-weight and stateless protocol to encode Remote Procedure Calls (RPCs). The specification defines data structures and processing rules. As such, the protocol is transport-agnostic and can be used with any type of message passing framework. The data structures are encoded using the JSON data format.

The protocol is versioned. Currently the versions 1.0 and 2.0 are specified. The LSP uses version 2.0 of JSON-RPC. Henceforth, the sections will focus on version 2.0 of the protocol.

JSON-RPC defines three types of messages:

- Requests
- Responses
- Notifications

, though notifications could be considered a special case of requests. The following section will provide details on each type.

Messages will be defined using TypeScript interfaces, in accordance with the LSP specification. A general message has the following interface:

```typescript
interface Message {
    jsonrpc: string;
}
```

Listing 2: General message interface

To define the other message types, additional TypeScript types are introduced:

- integer: A signed 32-bit integer aliased to the TypeScript type number.
- uinteger: An unsigned 32-bit integer aliased to the TypeScript type number.
- LSPArray:

Request objects have the following JSON format, as defined by the following Typescript interface:

```typescript
interface RequestMessage extends Message {
    method: string;
    params?: any;
    id: string | number | null;
}
```

Listing 3: Request object format

The jsonrpc field contains the version used and must be set to 2.0. The method field contains the name of the remote procedure to be called. Any parameters needed to call the remote function should be passed in the optional field params. At last, the id field identifies the message and thus allows correlation between requests and responses. Using fractional numbers is discouraged, as the floating point accuracy issue contradicts the identification of the id.

Each request must be answered by a response. The response object has the following format:

```typescript
interface ResponseMessage extends Message {
    result?: any;
    error?: ResponseError;
    id: string | number | null;
}

interface ResponseError {
    code: number;
    message: string;
    data?: any;
}
```

Listing 4: Response object format

Similar to the request object, the jsonrpc field must be set to 2.0. Either the result or error field will be set, depending on the success of the request. To correlate requests and responses, the response will contain the id of the request in the field id.

Error responses contain the error code, a short descriptive message and optional additional data. The specification defines the following error codes and messages:

Code	Message	Meaning
−32700	Parse error	The server received invalid JSON
−32600	Invalid Request	The request is not a valid request object
−32601	Method not found	The method does not exist or is not available
−32602	Invalid params	Invalid method parameters
−32603	Internal error	An internal error occurred while processing the request
−32000	Server error	Reserved for implementation-defined server errors

Table 2: Error codes and messages

In addition, the protocol allows custom error codes.

The final type of message is the notification. Notifications are a special type of request, as they do not contain an id. Notifications do not require a response from the server:

```typescript
interface NotificationMessage extends Message {
    method: string;
    params?: any;
}
```

Listing 5: Notification object format

Base protocol

The base protocol defines the general format, in which messages are sent between the client and server. Each message contains a headers and a content part, separated by \r\n.

The headers part contains at least one header field, which are key-value pairs separated by a colon. Each header field is terminated by a \r\n. The following headers are currently supported:

- Content-Length: A mandatory header that specifies the length of the content part in bytes.
- Content-Type: The Multipurpose Internet Mail Extensions (MIME) type of the content part. The default is application/vscode-jsonrpc; charset=utf-8 and no other charset is currently supported.

The content part is a JSON object, as defined by the JSON-RPC specification. The LSP specification differs from the JSON-RPC in the use of certain types. It restricts the numeric type used for `id` and `code` from `number` to `integer`, to disallow floating point identifiers.

Basic JSON structures

Many RPCs share common structures needed to communicate about language features. The following section introduces a few necessary ones.

The first important structure is the Uniform Resource Identifier (URI), as defined by rfc3986[11]. URIs are passed as strings and are used to uniquely identify files the language server and client work with.

To identify a position inside such a file, the protocol defines the `Position` structure:

```typescript
interface Position {
    line: uinteger;
    character: uinteger;
}
```

Listing 6: Position structure

Line and character define the position in the file starting at `0`.

The last structure is `TextDocumentItem`, which represents an editable text document:

```typescript
interface TextDocumentItem {
    uri: DocumentUri;
    languageId: string;
    version: integer;
    text: string;
}
```

Listing 7: TextDocumentItem structure

In addition to the unique identifier of the document, the `uri`, the structure contains the content of the document in the field `text`. The `languageId` field defines the language of the document. This is needed especially when the language server processes documents of different languages. Passing the `languageId` prevents re-interpreting the file extension. The possible language ids are defined in the specification and can be seen in Table 3 in full.

Lifecycle Messages

After defining the basic structures, this section introduces the first messages to control the lifecycle of the language server.

The first message the client sends the server, is the `initialize` message. No other communication will occur, until the server acknowledges the initialization with the response. The connection between client and server may only be initialized once. The `initialize` message has the following key fields:

```typescript
interface InitializeParams {
    rootUri?: DocumentUri;
    capabilities: ClientCapabilities;
    // ... more fields omitted
}
```

Listing 8: Initialize message parameters

The `rootUri` is an optional field that specifies the root path of the workspace, the client is working on. Setting the workspace e.g. enables finding virtual environments needed to scan the local code base. The next field `capabilities` is a complex structure of flags, which specify all capabilities the client has.

The server responds with the initialize result, which contains the server capabilities.

Afer receiving the response, the client must then acknowledge the response, by sending the initialized message. The server then has the option to e.g. register capabilities dynamically or run further initializations. Afterwards, the initialization process is finished.

After using the language server, the client can stop the server in a two step process. First, the client sends a shutdown message. The server then stops itself, but preserves its messaging capabilities to send a proper response. In the next step, the client sends the exit message, after which the server exits its process.

Document Synchronization

After introducing messages to establish and control the connection between server and client, this section focused on the next step, the document synchronization. The goal of these messages is to synchronize the server to the users actions. The server should provide capabilities for a file, while the user works on it.

The first message is the textDocument/didOpen notification. Sending this notification informs the server that the client opened the document. The message has the following structure:

```typescript
interface DidOpenTextDocumentParams {
  textDocument: TextDocumentItem;
}
```

Listing 9: DidOpenTextDocument message parameters

As soon as the file is closed, the client sends the textDocument/didClose notification. The message has the same structure as in Listing 9.

Other messages for synchronizing documents and notebooks exist and are omitted.

Language Features

This section displays messages to query language features. The LSP specification defines many different features, of which this section highlights the few relevant ones:

- Auto Completion
- Signature help
- Provide diagnostics data on the current file

The auto completion feature provides a list of completion items for a given position in a document. The completion items can be selected to provide more detailed information. On applying the selected completion item, the completion text is inserted into the document at the given position. The feature is split into the two messages textDocument/completion and completionItem/resolve.The first message queries the completion items for the given position in a text document. Each completion item only contains basic information. If further details are required, the second message completionItem/resolve asks the server to extend a single completion item with in depth details.

A completion item has the following structure:

```typescript
interface CompletionItem {
    label: string;
    insertText?: string;
    labelDetails?: CompletionItemLabelDetails;
    detail?: string;
    documentation?: string | MarkupContent;
    kind?: CompletionItemKind;
    tags?: CompletionItemTag[];
    deprecated?: boolean;
    preselect?: boolean;
    sortText?: string;
    filterText?: string;
    data?: any;
    // ... more fields omitted
}
```

Listing 10: CompletionItem structure

The first key property is label. The label is the main text to be inserted into the document and provides the full symbol. In case the document already contains a prefix of the label in the given location, the inserted text must not contain the prefix. In these cases the label property remains untouched, but insertText is set to the shortened text. insertText overrides the text to be inserted, but remains optional.

The labelDetails property contains additional information, such as the locator of the symbol to be inserted. The structure of the property can be found in Listing 11.

For accessibility, the completion item includes several human readable properties:

- detail provides a short description of the completion item.
- documentation provides a detailed description of the completion item. The description can be plain text or Markdown. The structure of the property can be found in Listing 13.

The next field kind specifies the type of the completion item. The type refers to the symbol kind, such as a "function", "field" or "string". The full list is defined in Listing 23.

Annotations can be found in the field tags. The only supported tag is Deprecated, as defined in Listing 12. It replaces the now deprecated field deprecated.

After receiving the completion items, the client can display the items in a list. The field preselect can mark a **single** completion item to be preselected in the list. Additionally, the client may filter and sort the items. In these cases it should use the fields filterText and sortText for theses tasks, instead of label.

Additional server specific data is stored in the field data. This information is needed by the server to resolve the completion item and should not be changed.

```typescript
interface CompletionItemLabelDetails {
    detail?: string;
    description?: string;
}
```

Listing 11: CompletionItemLabelDetails structure

```typescript
namespace CompletionItemTag {
    const Deprecated = 1;
}
type CompletionItemTag = 1;
```

Listing 12: CompletionItemTag enum

```typescript
interface MarkupContent {
    kind: MarkupKind;
    value: string;
}
```

Listing 13: MarkupContent structure

```typescript
namespace MarkupKind {
    const PlainText: 'plaintext' = 'plaintext',
    const Markdown: 'markdown' = 'markdown'
}
type MarkupKind = 'plaintext' | 'markdown';
```

Listing 14: MarkupKind enum

The Listing 13 structure includes markup content in the field value. The type of markup is defined in the field kind. Currently the two kinds **plain text** and **Markdown** are supported, as seen in Listing 14.

To request completions, the client then sends the textDocument/completion message. The request the following structure:

```typescript
interface CompletionParams {
    textDocument: TextDocumentIdentifier;
    position: Position;
    context?: CompletionContext;
    // ... more fields omitted
}
```

Listing 15: Completion message parameters

The request contains a position in the document textDocument. The optional context has further details on how the completion was triggered (triggerKind) and with which character it was triggered (triggerCharacter), as seen in Listing 16. Several trigger kinds are currently supported, such as invoked, triggered by a character, or re-triggered to expand an incomplete completion list (Listing 17).

```typescript
interface CompletionContext {
    triggerKind: CompletionTriggerKind;
    triggerCharacter?: string;
}
```

Listing 16: CompletionContext structure

```typescript
namespace CompletionTriggerKind {
    const Invoked: 1 = 1;
    const TriggerCharacter: 2 = 2;
    const TriggerForIncompleteCompletions: 3 = 3;
}
type CompletionTriggerKind = 1 | 2 | 3;
```

Listing 17: CompletionTriggerKind enum

The server then responds either with no result, directly with a list of completion items, or a completion list, as seen in Listing 18.

```typescript
interface CompletionList {
    isIncomplete: boolean;
    items: CompletionItem[];
    // ... more fields omitted
}
```

Listing 18: CompletionList structure

In addition to containing the completion items, the structure also may indicate that the list is incomplete and will be recomputed through the field isIncomplete.

Resolving a single completion item is done by sending the completionItem/resolve message. The message is the completion item to be resolved. The server then responds with the resolved completion item. The resolved completion item is a superset of the provided completion item.

The second feature is the signature help. The signature help provides information about the signature of the symbol at the specified position. A common use case is to display function and class parameters.

Sending the textDocument/signatureHelp request to the server returns the signature help. The request has a similar structure to Listing 15, as seen in Listing 19:

```typescript
interface SignatureHelpParams {
  textDocument: TextDocumentIdentifier;
  position: Position;
  context?: SignatureHelpContext;
  // ... more fields omitted
}
```

Listing 19: SignatureHelp message parameters

The context field contains additional information about the context in which the signature help was triggered.

The server responds with the signature help in the structure of Listing 20:

```typescript
interface SignatureHelp {
  signatures: SignatureInformation[];
  activeSignature?: uinteger;
  activeParameter?: uinteger;
}
```

Listing 20: SignatureHelp structure

The symbol could have several signatures, of which each is represented by a SignatureInformation structure. Each single SignatureInformation has a label, to be shown in the UI, as well as the documentation regarding the whole item. Single parameters can be retrieved through the parameters field. When typing a function call, the currently typed parameter becomes the active parameter in the SignatureInformation (activeParameter). The parameters each come with a label and documentation field. The label either is a string, or a range referring to the label of the outer signature.

The signature help structure is defined in Listing 21, while the single parameter information is provided in Listing 22.

```typescript
interface SignatureInformation {
  label: string;
  documentation?: string | MarkupContent;
  parameters?: ParameterInformation[];
  activeParameter?: uinteger;
}
```

Listing 21: SignatureInformation structure

```typescript
interface ParameterInformation {
  label: string | [uinteger, uinteger];
  documentation?: string | MarkupContent;
}
```

Listing 22: ParameterInformation structure

2.8 Related Work

3 Design

3.1 Interrupt-based meta-strategy

3.2 Language Server Protocol aided generation

3.3 Dataset design

4 Implementation

4.1 Interrupt-based meta-strategy

4.2 Language Server Protocol aided generation

4.3 Dataset design

5 Results

5.1 Collection method

5.2 Possible configurations

5.3 Results

6 Evaluation

6.1 Research Questions

6.2 Evaluation difficulties

6.3 Evaluation

7 Conclusion

7.1 Recap

7.2 Outlook

Acronyms

BOS: Begin of Sequence

BPE: Byte Pair Encoding

CLM: Causal Language Models

Code LLM: Large Language Model for Code

EOS: End of Sequence

IDE: Integrated Development Environment

JSON: JavaScript Object Notation

JSON-RPC: JavaScript Object Notation-Remote Procedure Call

LLM: Large Language Models

LM: Language Model

LSP: Language Server Protocol

MIME: Multipurpose Internet Mail Extensions

PAD: Padding

RAG: Retrieval Augmented Generation

RPC: Remote Procedure Call

URI: Uniform Resource Identifier

Bibliography

[1] "Speech and Language Processing." Accessed: Jun. 04, 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/°

[2] "ChatGPT." Accessed: Jun. 10, 2024. [Online]. Available: https://chatgpt.com/°

[3] "https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf." Accessed: Jun. 10, 2024. [Online]. Available: https://home.csulb.edu/~cwallis/382/readings/482/mccolloch.logical.calculus.ideas.1943.pdf°

[4] L. Weng, "Controllable Neural Text Generation." Accessed: Jun. 12, 2024. [Online]. Available: https://lilianweng.github.io/posts/2021-01-02-controllable-text-generation/°

[5] T. B. Brown *et al.*, "Language Models Are Few-Shot Learners." Accessed: Jun. 12, 2024. [Online]. Available: http://arxiv.org/abs/2005.14165°

[6] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2201.11903°

[7] S. Yao *et al.*, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." Accessed: Oct. 10, 2023. [Online]. Available: http://arxiv.org/abs/2305.10601°

[8] S. Rabanser, O. Shchur, and S. Günnemann, "Introduction to Tensor Decompositions and Their Applications in Machine Learning." Accessed: Jun. 20, 2024. [Online]. Available: http://arxiv.org/abs/1711.10781°

[9] "IEEE Standards Association." Accessed: Jun. 20, 2024. [Online]. Available: https://standards.ieee.org/°

[10] "BFloat16: The Secret to High Performance on Cloud TPUs." Accessed: Jun. 20, 2024. [Online]. Available: https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus°

[11] T. Berners-Lee, R. T. Fielding, and L. M. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," Jan. 2005. doi: 10.17487/RFC3986°.

Listings

List of Figures

Figure 1: IEEE 754 half-precision floating point number: float16 10
Figure 2: Brain Floating Point Format: bfloat16 10

List of Tables

Table 1: Relevant PyTorch data types 10
Table 2: Error codes and messages 15
Table 3: Possible language ids defined in the LSP specification 30

List of Code

Listing 1: Loading a pre-trained model 11
Listing 2: General message interface 14
Listing 3: Request object format 14
Listing 4: Response object format 15
Listing 5: Notification object format 15
Listing 6: Position structure 16
Listing 7: TextDocumentItem structure 16
Listing 8: Initialize message parameters 16
Listing 9: DidOpenTextDocument message parameters 17
Listing 10: CompletionItem structure 18
Listing 11: CompletionItemLabelDetails structure 18
Listing 12: CompletionItemTag enum 18
Listing 13: MarkupContent structure 19
Listing 14: MarkupKind enum 19
Listing 15: Completion message parameters 19
Listing 16: CompletionContext structure 19
Listing 17: CompletionTriggerKind enum 19
Listing 18: CompletionList structure 19
Listing 19: SignatureHelp message parameters 20
Listing 20: SignatureHelp structure 20
Listing 21: SignatureInformation structure 20
Listing 22: ParameterInformation structure 20
Listing 23: CompletionItemKind enum 31

List of Examples

Example 1: Language model predicting the last word of the famous English pangram. 3
Example 2: Natural language encoded into tokens. 4
Example 3: Enforcing "yes" or "no" answers 6
Example 4: Few-shot prompting 7
Example 5: Context injection 8

Appendix

Language	Language ID
ABAP	abap
Windows Bat	bat
BibTeX	bibtex
Clojure	clojure
Coffeescript	coffeescript
C	c
C++	cpp
C#	csharp
CSS	css
Diff	diff
Dart	dart
Dockerfile	dockerfile
Elixir	elixir
Erland	erlang
F#	fsharp
Git	git-commit and git-rebase
Go	go
Groovy	groovy
Handlebars	handlebars
HTML	html
Ini	ini
Java	java
JavaScript	javascript
JavaScript React	javascriptreact
JSON	json
LaTeX	latex
Less	less
Lua	lua
Makefile	makefile
Markdown	markdown
Objective-C	objective-c
Objective-C++	objective-cpp
Perl	perl
Perl 6	perl6
PHP	php
Powershell	powershell
Pug	jade
Python	python
R	r
Razor	razor
Ruby	ruby
Rust	rust
SCSS	scss or sass
Scala	scala
ShaderLab	shaderlab
Shellscript	shellscript
SQL	sql
Swift	swift
TypeScript	typescript
TypeScript React	typescriptreact
TeX	tex
Visual Basic	vb
XML	xml
XSL	xsl
YAML	yaml

Table 3: Possible language ids defined in the LSP specification

```typescript
namespace CompletionItemKind {
    const Text: 1;
    const Method: 2;
    const Function: 3;
    const Constructor: 4;
    const Field: 5;
    const Variable: 6;
    const Class: 7;
    const Interface: 8;
    const Module: 9;
    const Property: 10;
    const Unit: 11;
    const Value: 12;
    const Enum: 13;
    const Keyword: 14;
    const Snippet: 15;
    const Color: 16;
    const File: 17;
    const Reference: 18;
    const Folder: 19;
    const EnumMember: 20;
    const Constant: 21;
    const Struct: 22;
    const Event: 23;
    const Operator: 24;
    const TypeParameter: 25;
}
type CompletionItemKind = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25;
```

Listing 23: CompletionItemKind enum

QR code



Scan the QR code to access the source code of this thesis:
https://github.com/data-niklas/master_thesis°